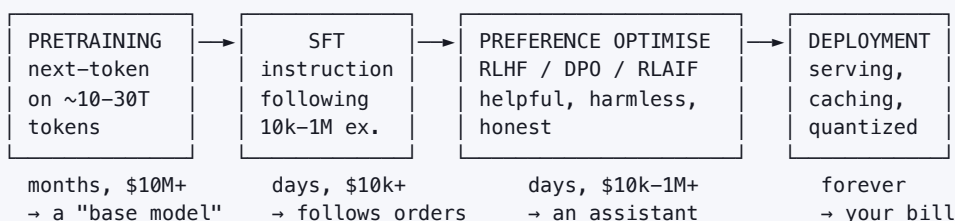


Core Concepts — How an LLM Is Built

Part 04 — 📖 Large Language Models · 04-llm/notes/01-how-llms-are-built.md

1. The whole pipeline on one page



The crucial distinction: a *base model* completes text. An *instruct/chat model* has been post-trained to follow instructions and refuse some requests. They behave completely differently, and most confusion about "why is the model doing that" comes from confusing the two.

2. Phase 1 — Pretraining

Objective: predict the next token. That's it. $L = -\sum \log P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$. It is the categorical cross-entropy you met in Module 01 [01-ml-foundations], applied at every position.

Why such a simple objective produces such general behaviour: to predict the next token well across the whole internet, you must implicitly model syntax, facts, arithmetic, code semantics, reasoning chains, authorial style, and the structure of dialogue. Compression is understanding, at least operationally.

Data

Modern pretraining corpora are 10–30+ trillion tokens: web crawl (Common Crawl, refined into FineWeb-class datasets), code (GitHub), books, papers (arXiv, PubMed), Wikipedia, curated Q&A, and increasingly **synthetic data** generated by earlier models.

The unglamorous truth: **data quality work is most of the value**. Deduplication (exact and near-duplicate), quality classifiers, toxicity and PII filtering, decontamination against eval benchmarks, and domain mixing ratios move benchmark scores more than architecture tweaks do.

Decontamination matters to you: if a benchmark's test set leaked into pretraining, the score is meaningless. This is the Module 01 [01-ml-foundations] test-set lesson at civilisational scale. See Module 15 [15-ai-vals].

Compute

FLOPs $\approx 6 \times N \times D$ for a dense model, where N = parameters and D = training tokens. (Forward is $\sim 2ND$, backward $\sim 4ND$.) This one formula lets you sanity-check any training claim you read.

3. Scaling laws — the most useful theory in the field

Kaplan et al. (2020): loss falls as a smooth power law in parameters, data, and compute. Performance is *predictable* — which is why labs can commit \$100M to a training run before it starts.

Chinchilla (Hoffmann et al., 2022) corrected the recipe. Given a fixed compute budget, the compute-optimal split is roughly **20 tokens per parameter**. GPT-3 (175B params, 300B tokens) was badly undertrained; a 70B model trained on 1.4T tokens beat it.

The 2023+ correction to the correction: Chinchilla optimises *training* compute. If you will serve a model billions of times, you should deliberately **overtrain a smaller model** — it costs more to train and far less to run. That is why 7B–70B models are now trained on 15T+ tokens, far past "compute-optimal".

Regime	Optimise for	Result
Research / frontier push	training compute	Chinchilla-optimal, ~ 20 tok/param
Production serving	inference cost	overtrain small models, 100–2000+ tok/param

Emergence: some capabilities appear to switch on abruptly at scale. Treat this claim with care — a substantial line of work (Schaeffer et al., 2023) argues much apparent emergence is an artefact of discontinuous metrics like exact-match. Use continuous metrics before you claim a phase transition.

4. Phase 2 — Supervised fine-tuning (SFT)

Train on curated (instruction, ideal response) pairs, using the same next-token loss but masking the loss on the prompt tokens. Typically 10k–1M examples.

This is where the **chat template** enters: special tokens delimiting system/user/assistant turns. Get the template wrong at inference — wrong special tokens, missing BOS, wrong role names — and quality collapses in ways that look like "the model got dumber". **Always use the tokenizer's `apply_chat_template`**.

Quality beats quantity, decisively. LIMA showed 1,000 carefully curated examples outperforming 50,000 noisy ones.

5. Phase 3 — Preference optimisation

SFT teaches *format*. Preference optimisation teaches *judgment* — which of two acceptable answers is better.

RLHF (the original)

1. Collect human comparisons: given a prompt and two responses, which is better?
2. Train a **reward model** to predict that preference.
3. Optimise the LLM against the reward model with PPO, with a KL penalty pinning it near the SFT model.

The KL penalty is essential: without it the policy finds degenerate text that scores highly on the reward model and is useless to humans — **reward hacking**, a textbook Goodhart failure.

DPO and friends (what most teams actually use now)

Direct Preference Optimization proves you can skip the explicit reward model and RL loop: a clever reparameterisation turns preference learning into a simple classification loss on preference pairs. Far simpler, far cheaper, far more stable. Variants: IPO, KTO (needs only binary good/bad, not pairs), ORPO, SimPO, GRPO (which powers much of the recent reasoning-model work).

Constitutional AI / RLAI

Replace (most) human labels with model-generated critiques against a written set of principles — a "constitution". Cheaper, more consistent, and auditable: the principles are a document you can review, which matters enormously for Module 12 [12-ai-governance].

What alignment costs you

- **Alignment tax:** some raw capability is traded for safety and format compliance.
- **Sycophancy:** models learn that agreeing with the user scores well. A direct consequence of optimising human approval, and a live problem.
- **Over-refusal:** an obvious failure mode, and one that evals must measure explicitly.

6. Reasoning models — the newer phase

A distinct post-training track trains models to produce long internal reasoning before answering, typically with reinforcement learning against verifiable rewards (math, code, tests that either pass or fail). Practical implications:

- You are now buying **inference-time compute**: more thinking tokens, better answers, higher latency and cost.
 - "Reasoning effort" becomes a tunable product parameter, not just a model choice.
 - Chain-of-thought prompting matters less when reasoning is baked in — and over-prompting can hurt.
 - Scaling test-time compute is a genuinely different axis from scaling parameters.
-

7. Context windows and what they actually buy

Context has grown from 2k (GPT-3) to 128k–1M+ tokens. Enabled by RoPE scaling (position interpolation, NTK-aware scaling, YaRN), efficient attention (FlashAttention, sliding-window), and a lot of infrastructure.

But: - Attention cost grows quadratically; KV cache grows linearly. Long context is *expensive*, not free. - **"Lost in the middle"**: retrieval accuracy dips for information in the middle of a long context. Put the critical material at the beginning or the end. - **Context rot**: as context fills with marginally-relevant material, quality degrades. More context is not monotonically better. - Needle-in-a-haystack tests are necessary but nowhere near sufficient — they test retrieval, not reasoning over the retrieved material.

Design conclusion: a long context window does not make RAG [08-rag] obsolete. It changes the trade-off. Retrieval is still cheaper, more current, more auditable, and more precise.

8. In-context learning and prompting

The model learns from examples *in the prompt*, with no weight updates. Mechanistically this is linked to **induction heads** — attention heads that implement "find an earlier occurrence of the current token and copy what followed it".

What reliably helps: - **Clear role and task statement**, then constraints, then the data. Put long reference material *first*. - **Few-shot examples** — especially for format. 3–5 examples usually captures most of the gain. - **Chain-of-thought** ("think step by step") for multi-step problems, on non-reasoning models. - **Structured output** — give a schema; better, use constrained decoding so it's guaranteed. - **Delimiters** (XML tags, markdown fences) to separate instructions from untrusted data. This is also a security control — see Module 13 [13-ai-security]. - **Prefill / response priming** where the API supports it.

What is oversold: elaborate personas, threats and bribes, and long incantations. Measure everything (Module 15 [15-ai-evals]); prompt folklore has a short half-life.

9. Failure modes you must design around

Failure	Why it happens	Mitigation
Hallucination	trained to produce <i>plausible</i> continuations, not to know what it doesn't know	RAG with citations, abstention instructions, verification passes, evals
Sycophancy	preference training rewards agreement	neutral prompt framing, don't leak your preferred answer, adversarial evals
Prompt sensitivity	tiny wording/format changes shift outputs	pin prompts, version them, regression-test them
Context rot	quality degrades as irrelevant context accumulates	aggressive retrieval, compaction, shorter working context
Knowledge cutoff	frozen training data	retrieval for anything time-sensitive; never ask "what year is it"
Tokenization artefacts	see Module 03 [03-nlp]	don't ask for character-level work; use tools for arithmetic
Non-determinism	sampling, and batching effects even at temperature 0	seeds where supported, tolerance-based tests, never assert exact equality
Jailbreaks / injection	instructions and data share one channel	Module 13 [13-ai-security]

10. Forward links

Idea here	Where it returns
Tool use in post-training	Module 06 — agents
Context limits and rot	Module 08 — why RAG persists
Reward hacking	Module 15 — why your eval metric will be gamed
Constitutional principles	Module 12 — governance artefacts
Quantization, serving	Module 16 — the tech stack
Model families and selection	Module 11 — the model landscape